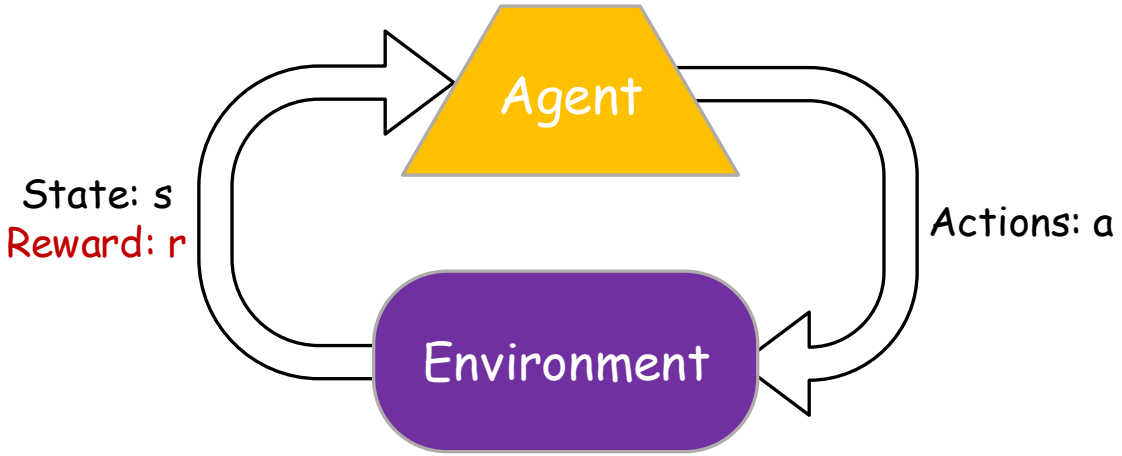
Reinforcement Learning

CHAPTER 21

Adapted from slides by Dan Klein, Pieter Abbeel, David Silver, and Raj Rao

Reinforcement Learning



- Basic idea:
  - Receive feedback in the form of rewards
  - Agent's utility is defined by the reward function
  - Must (learn to) act so as to maximize expected rewards
  - All learning is based on observed samples of outcomes!

Reinforcement Learning

- Still assume a Markov decision process (MDP):
  - A set of states s S
  - A set of actions (per state) A
  - A model $T(s, a, s')$
  - A reward function $R(s, a, s')$
- Still looking for a policy $\pi(s)$

- New twist: don't know T or R
  - We don't know which states are good or what the actions do
  - Must actually try actions and states out to learn

Passive Reinforcement Learning

- Simplified task: policy evaluation
  - Input: a fixed policy $\pi(s)$
  - You don't know the transitions $T(s, a, s')$
  - You don't know the rewards $R(s, a, s')$
  - Goal: learn the state values - V^{π}

- In this case:
  - No choice about what actions to take
  - Just execute the policy and learn from experience
  - **This is NOT offline planning! You actually take actions in the world.**

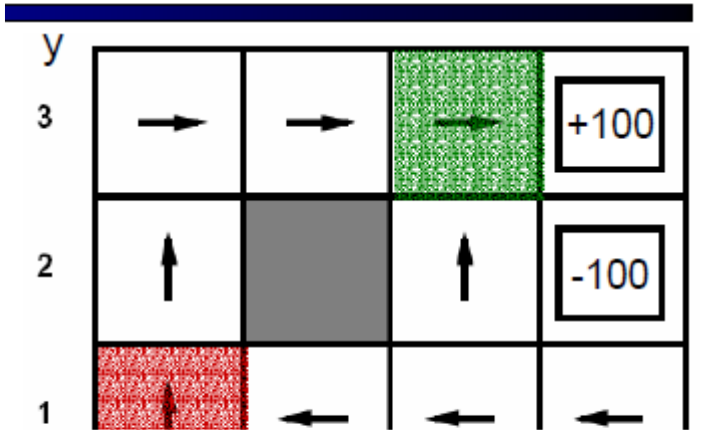
Episodes/Trials

- Episodes/trials

(1,1) up -1	(1,1) up -1
(1,2) up -1	(1,2) up -1
(1,2) up -1	(1,3) right -1
(1,3) right -1	(2,3) right -1
(2,3) right -1	(3,3) right -1
(3,3) right -1	(3,2) up -1
(3,2) up -1	(4,2) exit -100
(3,3) right -1	(done)
(4,3) exit +100	
(done)	

Empirical Policy Evaluation

Estimation



Method 1 - Adaptive Dynamic Programming

- **Idea**
  - Learn the model empirically
  - Solve the MDP as if the learned model were correct
  - Model based learning
- **Empirical model learning**
  - Simplest case
    - Count the outcomes for each s, a
    - Normalize to give estimate of $T(s, a, s')$
    - Discover $R(s, a, s')$ the first time we experience (s, a, s')
  - More complex learners are possible, if more domain knowledge is available

Method 1 - Adaptive Dynamic Programming (2)

- Estimate the transitions $T(s, \pi(s), s')$ and rewards $R(s, \pi(s), s')$ from the episodes
- Provides a model for MDP
- Apply Bellman equations to evaluate the policy

$$V_0^{\pi}(s) = 0$$

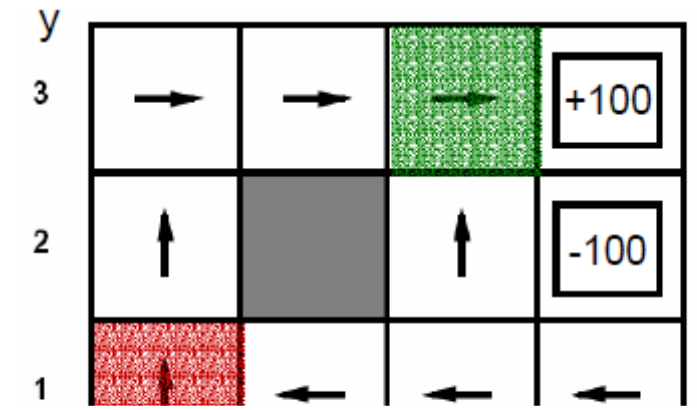
$$V_{k+1}^{\pi}(s) \leftarrow \sum_{s'} T(s, \pi(s), s')[R(s, \pi(s), s') + \gamma V_k^{\pi}(s')]$$

Method 1 - Adaptive Dynamic Programming (3)

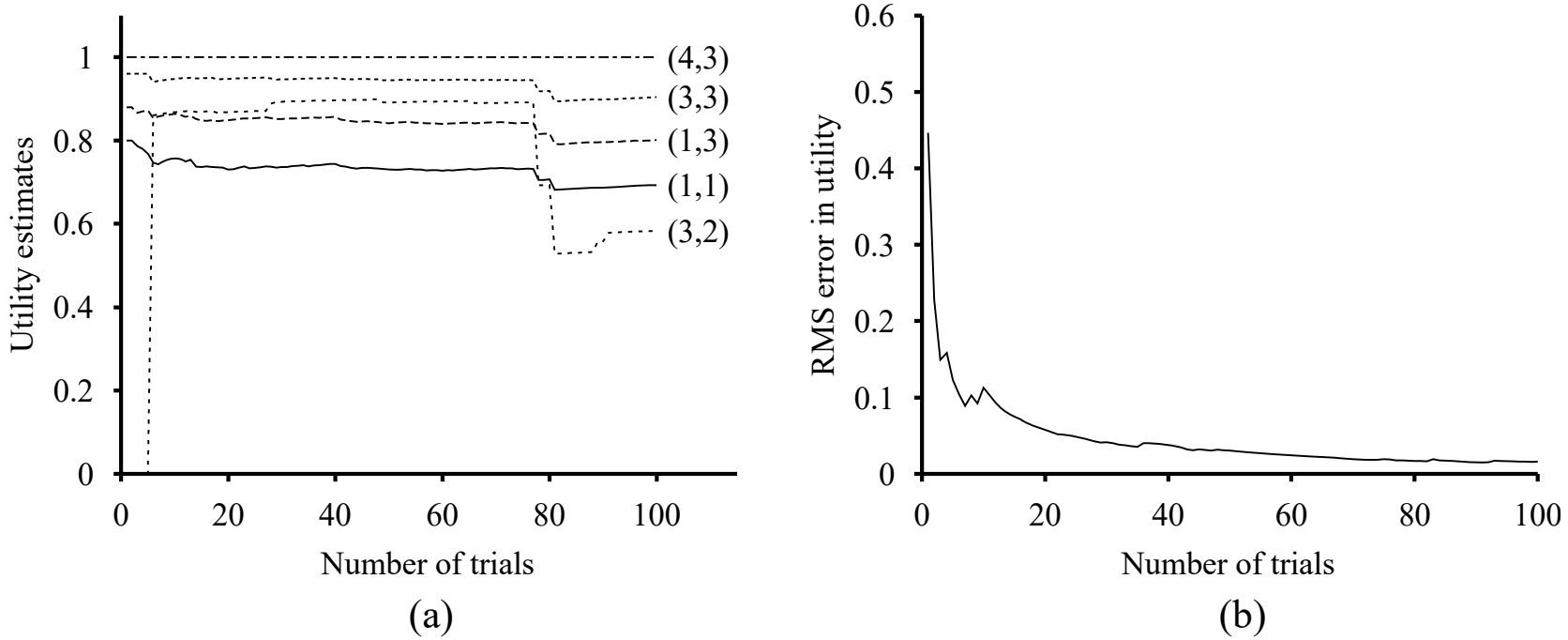
- Episodes

(1,1) up -1	(1,1) up -1
(1,2) up -1	(1,2) up -1
(1,2) up -1	(1,3) right -1
(1,3) right -1	(2,3) right -1
(2,3) right -1	(3,3) right -1
(3,3) right -1	(3,2) up -1
(3,2) up -1	(4,2) exit -100
(3,3) right -1	(done)
(4,3) exit +100	
(done)	

Estimation



Method 1 - Adaptive Dynamic Programming (4)



Method 1 - Adaptive Dynamic Programming (5)

- Intractable for large state spaces
- The ADP agent is limited only by its ability to learn the transition model.
  - ADP agent acts as if the learned model is correct – need not always be true.
- Can we turn it into a model free approach?

Method 2 - Direct Utility Estimation

- Goal: Compute values for each state under π
- Idea: Average together observed sample values
  - Act according to π
  - Every time you visit a state, write down what the sum of discounted rewards turned out to be
  - Average those samples
- This is called direct utility estimation/evaluation
  - Model free learning

Method 2 – Direct Utility Estimation

- Monte-Carlo Policy Evaluation
- To estimate $V^{\pi(s)}$
- Loop over episodes
  - The first time-step t that state s is visited in an episode
  - Increase the counter $N(s)$ - number of times s is visited in the episodes (can be multiple times in an episode)
  - Total return $T(s)$ – sum of the returns from each visit
  - Estimated return $\hat{V}^{\pi}(s) = T(s)/N(s)$
    - As $N(s) \to \infty$, $\hat{V}^{\pi}(s) \to V^{\pi(s)}$

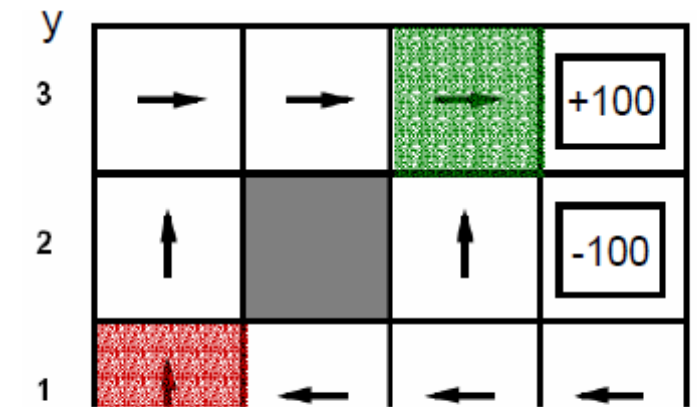
Method 2- Direct Utility Estimation

- Episodes/trials

Empirical Policy Evaluation

(1,1) up -1	(1,1) up -1
(1,2) up -1	(1,2) up -1
(1,2) up -1	(1,3) right -1
(1,3) right -1	(2,3) right -1
(2,3) right -1	(3,3) right -1
(3,3) right -1	(3,2) up -1
(3,2) up -1	(4,2) exit -100
(3,3) right -1	(done)
(4,3) exit +100	
(done)	

Estimation



Incremental Mean

- The mean $\mu_1, \mu_2, \ldots$ of a sequence $x_1, x_2, \ldots$ can be computed incrementally

$$\mu_k = \frac{1}{k}\sum_{i=1}^{k} x_i$$

$$= \frac{1}{k}\left(x_k + \sum_{i=1}^{k-1} x_i\right)$$

$$= \frac{1}{k}\left(x_k + (k-1)\mu_{k-1}\right)$$

$$= \mu_{k-1} + \frac{1}{k}\left(x_k - \mu_{k-1}\right)$$

Method 3 – Temporal Difference Learning

- Idea – Learn from every experience
  - Update $V(s)$ each time we experience a transition (s, a, s', r)
  - Likely outcomes s' will contribute to updates more often
- Temporal difference learning of values
  - Policy still fixed, still doing evaluation
  - Model-free – no knowledge of the MDP
  - Learns from incomplete episodes, bootstrapping
    - Update a guess towards a guess

Method 3 – Temporal Difference Learning

- Idea – Learn from every experience
  - Update $V(s)$ each time we experience a transition (s, a, s', r)
  - Likely outcomes s' will contribute to updates more often
- Temporal difference learning of values
  - Policy still fixed, still doing evaluation
  - $sample = R(s, \pi(s), s') + \gamma \hat{V}^{\pi}(s')$ - TD target
  - $\hat{V}^{\pi}(s) \leftarrow \hat{V}^{\pi}(s) + \alpha \left(sample - \hat{V}^{\pi}(s) \right)$
    - $sample - \hat{V}^{\pi}(s)$ - TD error

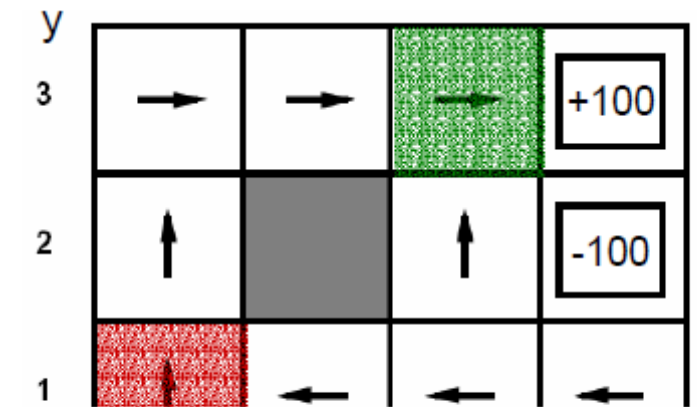
Method 3 - Temporal Difference Learning

- Episodes/trials

(1,1) up -1	(1,1) up -1
(1,2) up -1	(1,2) up -1
(1,2) up -1	(1,3) right -1
(1,3) right -1	(2,3) right -1
(2,3) right -1	(3,3) right -1
(3,3) right -1	(3,2) up -1
(3,2) up -1	(4,2) exit -100
(3,3) right -1	(done)
(4,3) exit +100	
(done)	

Empirical Policy Evaluation

Estimation



TD and ADP

- Both try to make local adjustments to utility estimates
- TD adjusts a state to agree to its observed successor
- ADP adjusts the state to agree to with all the successors that might occur, weighted by their probabilities
  - TD adjustments averaged over a large number of transitions will result in the same outcome